

Relatório SGRAI

Sprint 3

Realizada por:

1221352 – Bruno Ribeiro

1230430 – Eduardo Ferreira

1220839 – Vasco Teixeira

1221957 - João Oliveira

US 7.5.1 e US 7.5.4

Enunciado:

“As a healthcare staff member, I want to select a room by left clicking on the corresponding surgical table (object picking). The camera should instantly move horizontally so that it targets the room center point.”

“As a healthcare staff member, whenever I select a different room, I want the camera (and the spotlight, if applicable) to move smoothly instead of instantly. The animation can either be created or defined using some API, such as tween.js (four- and five-member groups only).”

Para a funcionalidade requerida é pretendida a implementação através do Tween.js de uma animação de deslocação para a cama pretendida, a animação deverá ser lenta de modo a demonstrar o seu efeito. O ponto final da deslocação deverá ser sobre a cama selecionada.

```
moveCameraToRoom(position, camera, activateSpotlightFlag) {
  console.log(activateSpotlightFlag);
  const [row, column] = this.cartesianToCell(position);

  const centerX = (column - this.size.width / 2.0 + 0.5) * this.scale.x;
  const centerZ = (row - this.size.height / 2.0 + 0.5) * this.scale.z;

  const cameraHeight = 5.0;

  const startPosition = {
    x: camera.object.position.x,
    y: camera.object.position.y,
    z: camera.object.position.z,
  };

  const endPosition = {
    x: centerX,
    y: cameraHeight,
    z: centerZ,
  };

  const lookAtTarget = new THREE.Vector3(centerX, 0, centerZ);

  var tween = new Tween(startPosition, true)
    .to(endPosition, 1500)
    .easing(TWEEN.Easing.Quadratic.InOut)
    .onUpdate(() => {
      camera.object.position.set(
        startPosition.x,
        startPosition.y,
        startPosition.z
      );
      camera.object.lookAt(lookAtTarget);
    })
    .onComplete(() => {
      camera.object.position.set(
        endPosition.x,
        endPosition.y,
        endPosition.z
      );
      camera.object.lookAt(lookAtTarget);
      if (activateSpotlightFlag) {
        this.activateSpotlight(position);
      }
    })
    .start();

  tween.update();
}
```

Figura 1 - Método moveCameraToRoom()

Como mencionado anteriormente foi utilizado o TWEEN.js onde são recolhidas as coordenadas do ponto inicial e final para a criação de uma animação e utilizados no método “moveCameraToRoom()”, para a detecção deste clique foram criadas estruturas “boxes” à volta de cada quarto onde em caso de um clique sobre as mesmas estas irão acionar um “EventListener” que utilizará posteriormente o “onClick()” responsável pela criação da câmera e spotlight responsável pela animação. Este método “moveCameraToRoom()” começa por calcular a posição final da câmara tendo em conta o quarto escolhido. Depois criamos um Tween que vai criar a animação suave pretendida e vai demorar um total 1,5 segundos até estar concluída. Caso esteja a ocorrer uma operação na sala é pretendido que um spotlight seja utilizado para criar iluminação na sala indicando que está a ocorrer uma operação. Para isso no método anterior incorpora-se também o método na imagem abaixo que cria um spotlight e ativa-o.

```
activateSpotlight(position) {  
    this.spotlight.position.set(position.x, position.y+5, position.z);  
  
    this.spotlight.target.position.set(position.x, position.y, position.z);  
  
    this.spotlight.color.set(0xADD8E6);  
  
    this.spotlight.intensity = 20;  
  
    this.spotlight.angle = Math.PI / 6;  
  
    this.spotlight.distance = 10;  
  
    console.log("Spotlight ativado para o paciente na posição:", position);  
}
```

Figura 2 - Método activateSpotlight()

Este método define a posição do spotlight(calculado no método anterior e é posteriormente passada por parâmetro neste método) e também os parâmetros da cor, intensidade o ângulo e a distância. Com estes dois métodos conseguimos concluir com êxito o que era pretendido para estas duas USs.

Para demonstração do resultado obtido, encontram-se abaixo uma imagem do antes e após clique na cama pretendida (com operação e sem operação), a animação da deslocação até á cama demora 1,5 segundos.

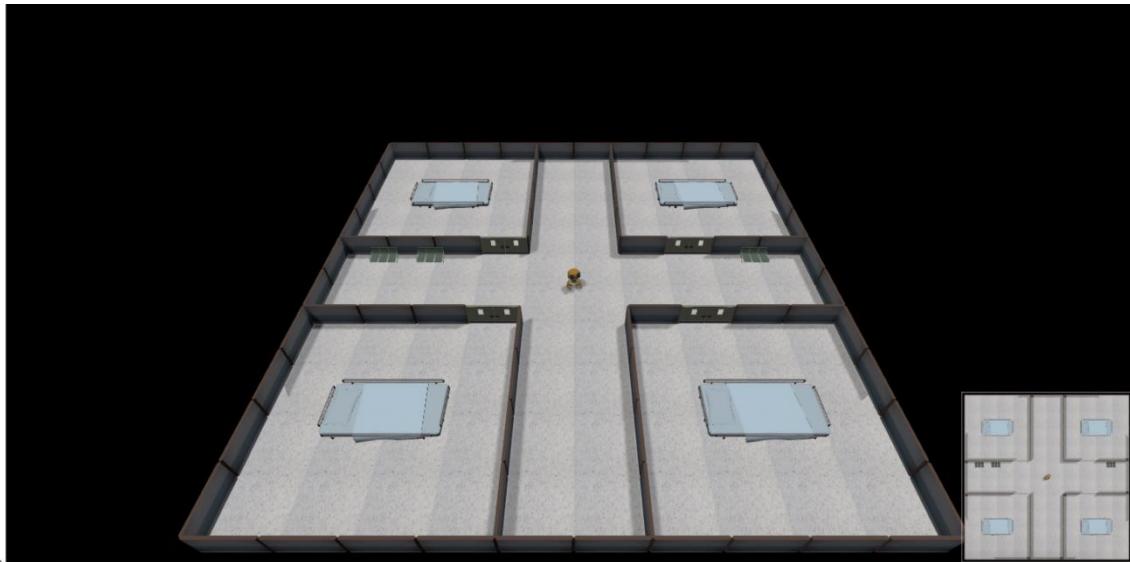


Figura 3– Posição da câmara padrão



Figura 4– Posição da câmara após clique numa sala sem operação a decorrer

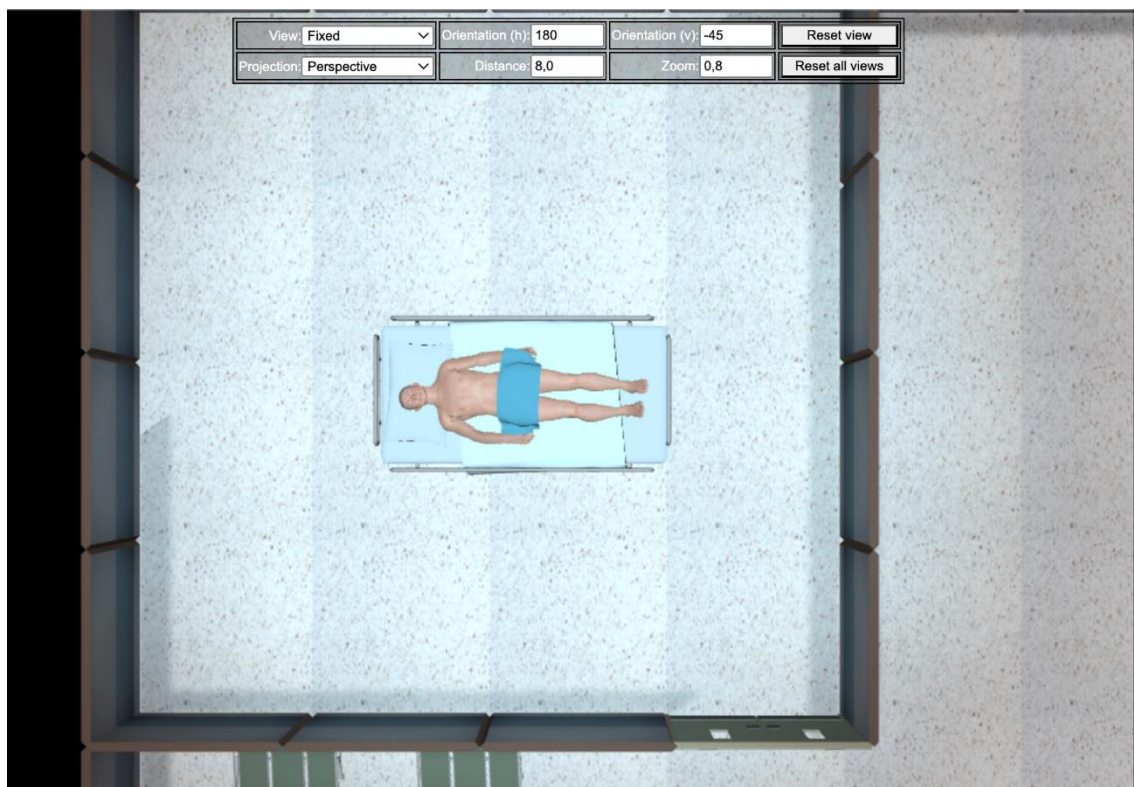


Figura 5– Posição da câmara após clique numa sala com operação a decorrer

US 7.5.2

“As a healthcare staff member, whenever I press the “i” key, I want to display/hide an overlay containing updated information about the selected room.”

Esta User Story tem como objetivo o sistema exibir ou ocultar um overlay contendo informações atualizadas sobre a sala selecionada. O processo deve ser acionado pela detecção da tecla “i” e, dependendo dessa ação, a sobreposição será exibida ou escondida. As informações que vão aparecer vão ser em tempo real, isto é, trazem as informações da sala e caso haja uma operação também traz informação da operação.

Para isso temos o método “toggleOverlay()” que é responsável por mostrar e ocultar o overlay. Ao mostrar colocamos um timeout para o sistema ter tempo de ir buscar as informações ao backend.

```
toggleOverlay () {  
  this.overlayVisible = !this.overlayVisible;  
  const overlayElement = document.getElementById("overlay");  
  if (this.overlayVisible) {  
    overlayElement.style.display = "none";  
  
    setTimeout(() => {  
      overlayElement.style.display = "block";  
      this.fetchRoomInfo(this.clickedObject);  
    }, 500);  
  } else {  
    overlayElement.style.display = "none"; // Esconde o overlay  
  }  
}
```

Figura 6 - Método toggleOverlay()

O método “fetchRoomInfo()” é responsável por saber qual o objeto interseável e identificar qual o ID da sala que está selecionada. Depois de acordo com a existência ao não de operação vai ao backend buscar apenas as informações que são necessárias e este método vai chamar o método que coloca a informação no overlay que queremos mostrar que é o “updateOverlayContent()”.

```

async fetchRoomInfo() {
    const intersects = this.raycaster.intersectObjects(this.scene3D.children, true);
    const clickedObject = intersects[0].object;
    console.log("Object clicked:", clickedObject.name);
    let surgeryRoomId = "";

    // Identificar o ID da Sala
    if (clickedObject.name.includes("2_2")) {
        surgeryRoomId = "SR001";
    } else if (clickedObject.name.includes("2_8")) {
        surgeryRoomId = "SR002";
    } else if (clickedObject.name.includes("8_2")) {
        surgeryRoomId = "SR003";
    } else if (clickedObject.name.includes("8_8")) {
        surgeryRoomId = "SR004";
    }

    try {
        // Buscar informações da Sala
        if (clickedObject.name.includes("Bed")) {
            const roomResponse = await fetch(`https://localhost:5001/api/surgeryRooms/${surgeryRoomId}`);
            const roomData = await roomResponse.json();
            this.updateOverlayContent(roomData);
        }

        // Buscar informações do Appointment para a Sala
        else if (clickedObject.name.includes("Patient")) {
            const appointmentResponse = await fetch(`https://localhost:5001/api/hospitalModel/currentAppointment/${surgeryRoomId}`);
            const appointmentData = await appointmentResponse.json();
            const patientData = await fetch(`https://localhost:5001/api/patients/${appointmentData.operationRequestDto.medicalRecordNumber}`);
            const patientJson = await patientData.json();
            appointmentData.patientName = patientJson.fullName;
            this.updateOverlayContent(appointmentData);
        }
    } catch (error) {
        console.error("Erro ao buscar informações:", error);
    }
}

```

Figura 7 - Método FetchRoomInfo()

O método “updateOverlayContent()” tem como objetivo colocar a informação no overlay tendo em conta se existe ou não operação. Caso não exista entra apenas no primeiro if, se existir operação entra também no segundo if e coloca também a informação da operação.

```

updateOverlayContent(data) {
  const overlayContent = document.getElementById("overlay-content");
  let contentHTML = "";

  if (data.id && data.equipment && data.roomCapacity && data.status && data.maintenanceSlots) {
    contentHTML += `
      <h2>Informações da Sala</h2>
      <p><strong>ID:</strong> ${data.id}</p>
      <p><strong>Equipment:</strong> ${data.equipment}</p>
      <p><strong>Capacity:</strong> ${data.roomCapacity}</p>
      <p><strong>Room Type:</strong> ${data.roomType.designation}</p>
      <p><strong>Maintenance Slots:</strong> ${data.maintenanceSlots}</p>
      <p><strong>Status:</strong> ${data.status}</p>
    `;
  }

  if (data.patientName) {
    contentHTML += `
      <h2>Informações da Sala</h2>
      <p><strong>ID:</strong> ${data.surgeryRoomDto.id}</p>
      <p><strong>Equipment:</strong> ${data.surgeryRoomDto.equipment}</p>
      <p><strong>Capacity:</strong> ${data.surgeryRoomDto.roomCapacity}</p>
      <p><strong>Room Type:</strong> ${data.surgeryRoomDto.roomType.designation}</p>
      <p><strong>Maintenance Slots:</strong> ${data.surgeryRoomDto.maintenanceSlots}</p>

      <h2>Informações do Appointment</h2>
      <p><strong>Patient:</strong> ${data.patientName}</p>
      <p><strong>Operation:</strong> ${data.operationRequestDto.operationType.name}</p>
      <p><strong>Status:</strong> ${data.dateAndTime}</p>
      <p><strong>Duration:</strong> ${data.operationRequestDto.operationType.estimatedDuration}</p>
    `;
  }

  if (contentHTML === "") {
    contentHTML = `<p>Nenhuma informação disponível para exibição.</p>`;
  }

  overlayContent.innerHTML = contentHTML;
  this.showOverlay();
}

```

Figura 8- Método updateOverlayContent()

Em seguida vamos colocar as imagens que demonstram as funcionalidades que foram pedidas. Na primeira imagem vamos ter um overlay que contém apenas informação da sala pois não está a ocorrer nenhuma operação, na segunda imagem vamos ter informação também da operação.

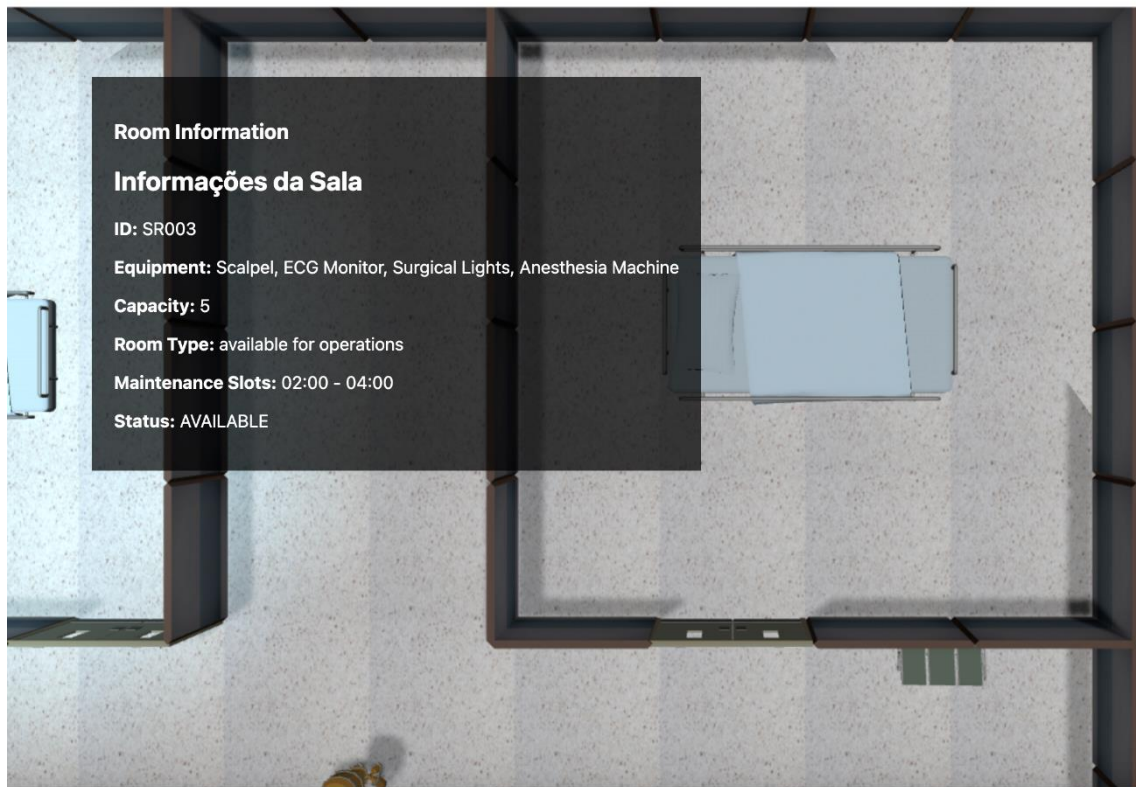


Figura 9 – Overlay com informação da sala apenas

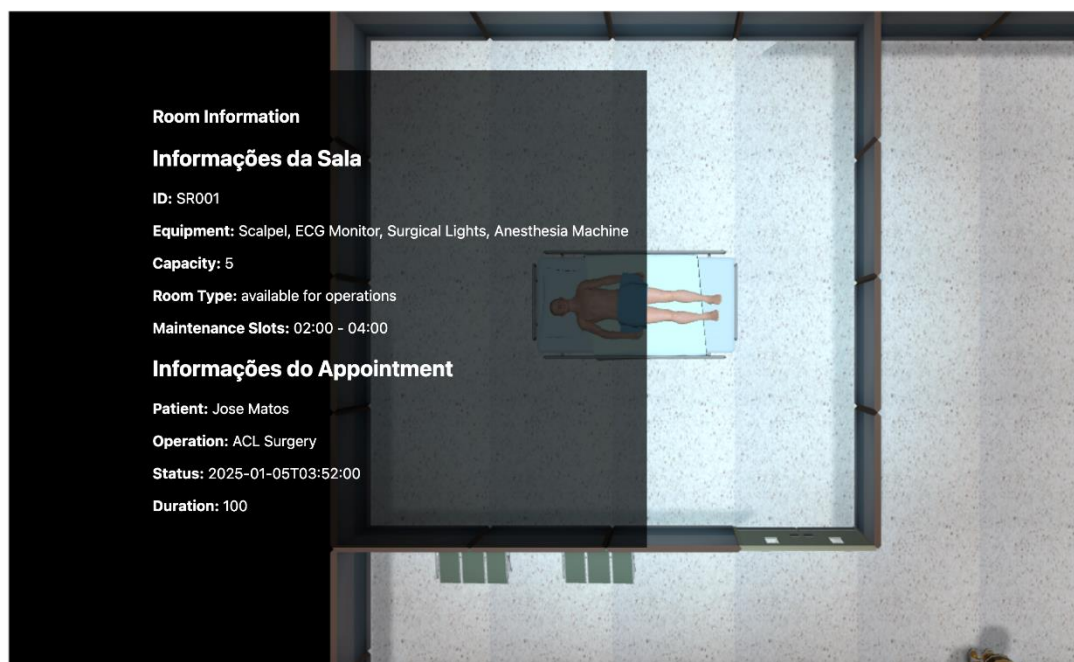


Figura 10 – Overlay com informação da sala e também da operação a decorrer